Etapa 8: ¿mejora el tiempo completo?

La generación distribuida redujo el tiempo de preparar la entrada en la etapa anterior. Ahora preguntamos si esa mejora se mantiene al incluir la factorización y devolver el resultado al proceso 0.

Una comparación con el mismo resultado

Centralizada: rango 0 genera A, empaqueta y distribuye columnas. Distribuida: cada proceso genera directamente sus columnas de la misma A. Ambas ejecutan right-looking MPI cíclico con vector local y reúnen L completa en rango 0. La referencia secuencial genera A y aplica right-looking con vector en rango 0.

Qué llamamos total en este experimento

$$T = T_{reserva} + T_{generacion} + T_{distribucion} + T_{factorizacion} + T_{recoleccion}$$

Medimos directamente el intervalo desde el comienzo de las reservas hasta que L queda completa en rango 0, con MPI ya iniciado. Las fases se registran consecutivamente en ese mismo proceso y suman su total por ejecución. También guardamos el máximo de las duraciones locales como control.

Quedan fuera el lanzamiento y cierre de MPI, las referencias de prueba, la validación, la liberación de memoria y escribir resultados. Es el flujo de procesamiento, no el tiempo externo del comando mpirun. Las reservas se repiten; malloc no toca todas las páginas, por lo que parte del costo de memoria se paga al acceder a ellas en las fases posteriores.

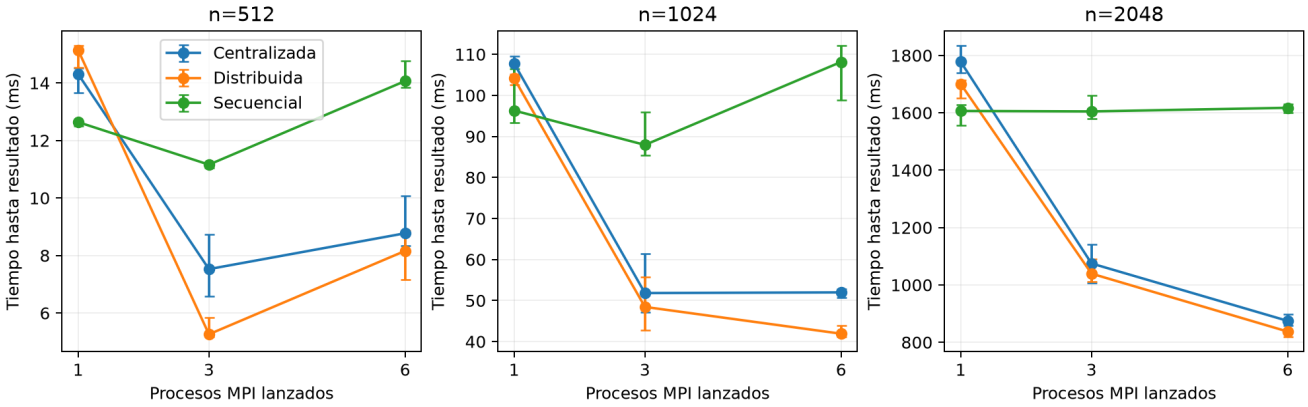
Por qué usar medianas

Calculamos la mediana de los totales observados. No sumamos medianas de fases para obtenerla. Para conocer el peso de una fase, dividimos su duración por el total de esa misma ejecución y luego calculamos la mediana de esas proporciones. Estas medianas tampoco tienen que sumar exactamente 100 %.

Protocolo

n=512,1024,2048; P=1,3,6; un calentamiento y seis repeticiones por método y configuración, usando las seis permutaciones de los tres métodos. Son 162 observaciones en una sesión WSL. Tamaños y P se recorren en orden creciente. Ryzen 5 2600, afinidad por núcleo, GCC -O3; semilla 2026. El secuencial corre dentro del lanzamiento MPI con los otros procesos esperando, condición que debe tenerse presente al interpretar su referencia.

Resultados del flujo integrado



Medianas y barras Q1-Q3. No son intervalos de confianza.

n	P	Central (ms)	Distrib. (ms)	Secuencial (ms)
512	3	7.53	5.26	11.16
512	6	8.77	8.16	14.06
1024	3	51.82	48.44	87.92
1024	6	51.98	41.91	108.15
2048	3	1074.82	1039.34	1604.65
2048	6	874.60	837.39	1617.31

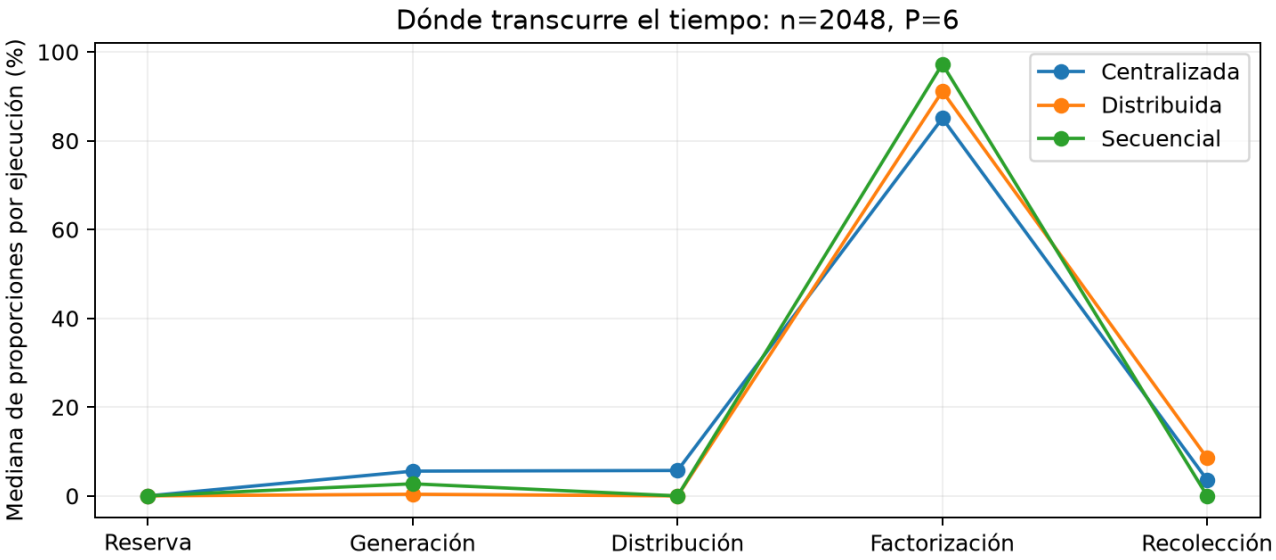
Para n=2048 y P=6: cambiar la preparación centralizada por distribuida da un cociente de 1.04x y una reducción de 4.3 % del tiempo mediano. Respecto del secuencial medido en esa configuración, el método distribuido alcanza 1.93x.

Lo que esta comparación permite afirmar

Comparamos resultados completos equivalentes y datos idénticos, sin combinar tiempos de campañas diferentes. La velocidad de la generación aislada no se traslada automáticamente al total: la factorización, la salida y las interacciones con memoria siguen costando tiempo.

No atribuimos diferencias de factorización exclusivamente al algoritmo, que es el mismo en ambas variantes MPI. La generación cambia el estado de caché, el primer acceso a memoria y cuándo llega cada proceso al kernel. Los tiempos de fases incluyen las esperas resultantes.

¿Dónde conviene trabajar ahora?



En el flujo distribuido de n=2048, P=6, la factorización representa una mediana del 91.1 % del tiempo observado y la recolección del 8.5 %. Estas proporciones sirven para priorizar el próximo experimento local; no predicen el comportamiento de la red del clúster.

Validación

Se aceptaron 162 observaciones. Máximo residuo proyectado: 1.611e-15; máxima diferencia relativa por elemento respecto del factor secuencial: 0.000e+00. Se comprobó el factor completo contra una referencia secuencial en cada repetición. El residuo Frobenius completo se usa en las pruebas pequeñas (hasta n=128).

La comprobación proyectada usa ocho vectores de signos y semilla 91827, fuera del tiempo útil, y su duración se registra por separado. No es un certificado ni el residuo Frobenius. Las pruebas incluyen n=1,2,3,7,31,128 y procesos sin columnas.

Próxima hipótesis y límites

Si el cálculo concentra el tiempo, el próximo cambio debe estudiar la factorización y el reparto, empezando por bloques de columnas cíclicos. Una distribución bloque-cíclica modifica propietarios; un algoritmo Cholesky por bloques modifica las operaciones. Debemos mantener esa distinción y cambiar una decisión por experimento.

Una sola sesión y seis repeticiones no establecen un ganador universal. Falta confirmar configuraciones elegidas en sesiones independientes, ampliar tamaños y medir lanzamiento externo si la aplicación crea procesos por cada matriz. El banco conserva matrices de referencia fuera del cronómetro: su consumo de memoria no es el mínimo del algoritmo.

Fuentes: src/compare_application.c; scripts/comparar_aplicacion.sh; scripts/analizar_aplicacion.py. Evidencia: resultados/aplicacion_local_20260922. Implementación y documentación con asistencia de IA, comprobadas mediante ejecución local.